

視覚メディア処理Ⅰ 第3回レポート

提出日: 2026 年 6 月 3 日

学籍番号: 2611193

氏名: 中井 平蔵

1. 使用画像と設定

15 枚の画像を、枚数の異なる 4 種類のセットに分けた。分割方法を表 1 に、使用画像を図 1 に示す。

表 1 データセットの分割方法

セット	枚数	使用画像
Set_A_3	3	0.jpg-2.jpg
Set_B_7	7	0.jpg-6.jpg
Set_C_11	11	0.jpg-10.jpg
Set_D_15	15	0.jpg-14.jpg

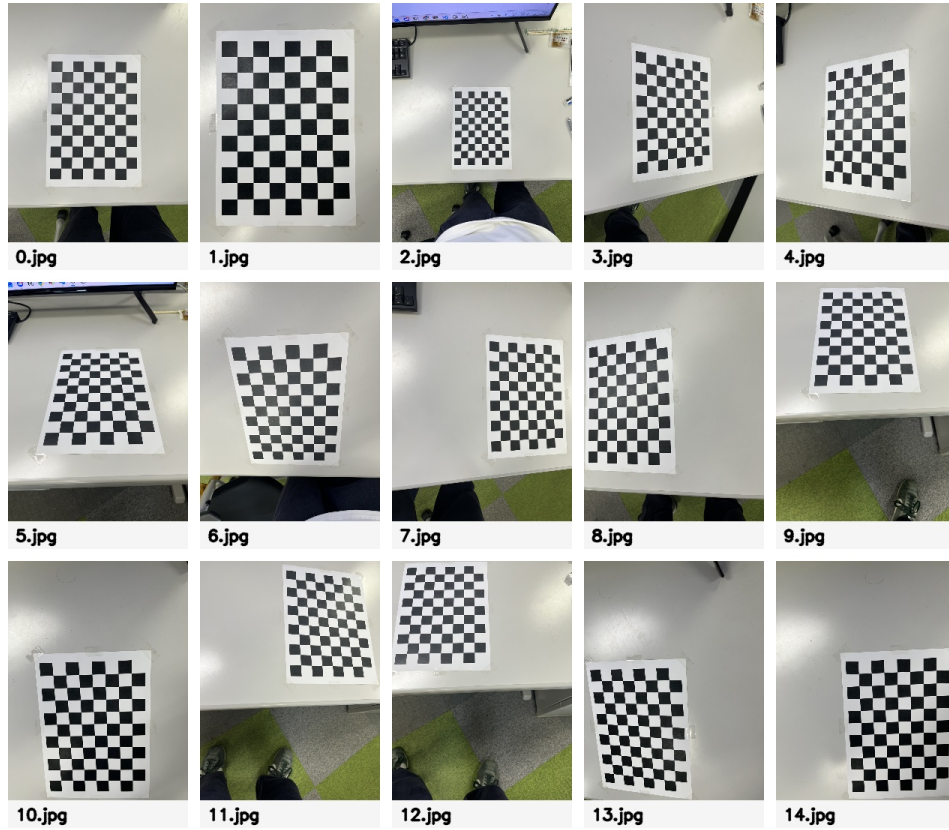


図1 使用した15枚のキャリブレーション画像

各画像では、OpenCVによりチェッカーボードの内側交点を検出した。今回使用したチェッカーボードは、横方向7点、縦方向10点の内側交点を持つため、検出パターンサイズを

$$(7, 10)$$

とした。1マスの実寸は **24 mm** として設定した。したがって、チェッカーボード平面上の3次元点は、左上の内側交点を原点として

$$\mathbf{X}_{ij} = (24i, 24j, 0)^T \text{ [mm]}$$

とした。ただし、 $i = 0, \dots, 6$ 、 $j = 0, \dots, 9$ である。

キャリブレーションの実装には OpenCV を用いた。チェッカーボード交点の検出には `cv2.findChessboardCornersSB` を使用し、内部パラメータ推定には `cv2.calibrateCamera` を使用した。

2. カメラモデル

カメラの内部パラメータ行列を

$$K = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix}$$

とする。ここで、 f_x, f_y はピクセル単位の焦点距離、 c_x, c_y は主点座標である。

歪み係数は OpenCV の標準的なモデルに従い、

$$(k_1, k_2, p_1, p_2, k_3)$$

として推定した。 k_1, k_2, k_3 は半径方向歪み、 p_1, p_2 は接線方向歪みを表す。

再投影誤差は、画像から検出した交点 $\mathbf{x}_i$ と、推定したカメラパラメータで 3 次元点を画像上に投影した点 $\hat{\mathbf{x}}_i$ の距離である。RMS 誤差は

$$\text{RMS} = \sqrt{\frac{1}{N} \sum_{i=1}^N \|\mathbf{x}_i - \hat{\mathbf{x}}_i\|^2}$$

で計算される。単位はピクセルであり、小さいほど検出点と再投影点の差が小さいことを示す。

3. 実行結果

各セットで得られた内部パラメータと再投影誤差を表 2 に示す。OpenCV RMS は `cv2.calibrateCamera` が返す全体の RMS 再投影誤差である。Mean image RMS は、各画像ごとに計算した RMS 誤差の平均である。

表 2 画像セットごとの内部パラメータと再投影誤差

セット	枚数	OpenCV RMS	Mean image RMS	f_x	f_y	(c_x, c_y)
Set_A_3	3	0.218	0.202	1235.763	1234.375	(558.250, 727.926)
Set_B_7	7	0.295	0.283	1141.844	1140.028	(566.494, 732.188)
Set_C_11	11	0.344	0.331	1143.386	1141.363	(562.532, 735.351)
Set_D_15	15	0.425	0.416	1146.044	1143.997	(560.469, 731.875)

歪み係数を表 3 に示す。

表 3 画像セットごとの歪み係数

セット	k_1	k_2	p_1	p_2	k_3
Set_A_3	0.265	-1.831	-3.67×10^{-4}	5.29×10^{-4}	4.219
Set_B_7	0.233	-1.350	-3.86×10^{-4}	3.66×10^{-4}	2.609
Set_C_11	0.181	-0.883	4.96×10^{-4}	-6.25×10^{-4}	1.528
Set_D_15	0.098	-0.260	-8.26×10^{-4}	-8.63×10^{-4}	0.274

各画像における最大の画像別 RMS 誤差は、Set_A_3 で 0.311 px、Set_B_7 で 0.355 px、Set_C_11 で 0.508 px、Set_D_15 で 0.533 px であった。すべて 1 ピクセル未満であり、検出した交点と再投影点のずれは小さい。

4. 再投影誤差の可視化

再投影誤差が画像上でどの程度ずれているかを確認するため、検出点と再投影点を同じ画像上に描画した。緑の点が OpenCV で検出した交点、赤の点が推定パラメータにより再投影された点である。

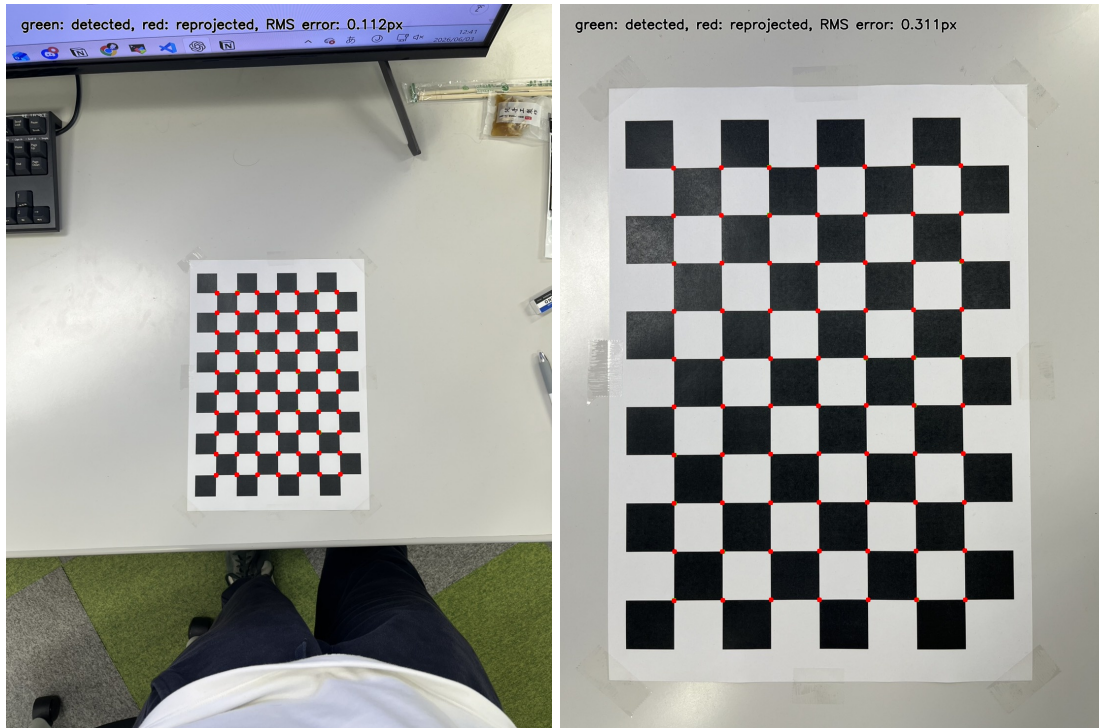


図 2 Set_A_3 の最小誤差画像（左: 2.jpg, 0.112 px）と最大誤差画像（右: 1.jpg, 0.311 px）



図3 Set_B_7 の最小誤差画像（左: 2.jpg, 0.107 px）と最大誤差画像（右: 5.jpg, 0.355 px）



図4 Set_C_11 の最小誤差画像（左: 2.jpg, 0.128 px）と最大誤差画像（右: 9.jpg, 0.508 px）

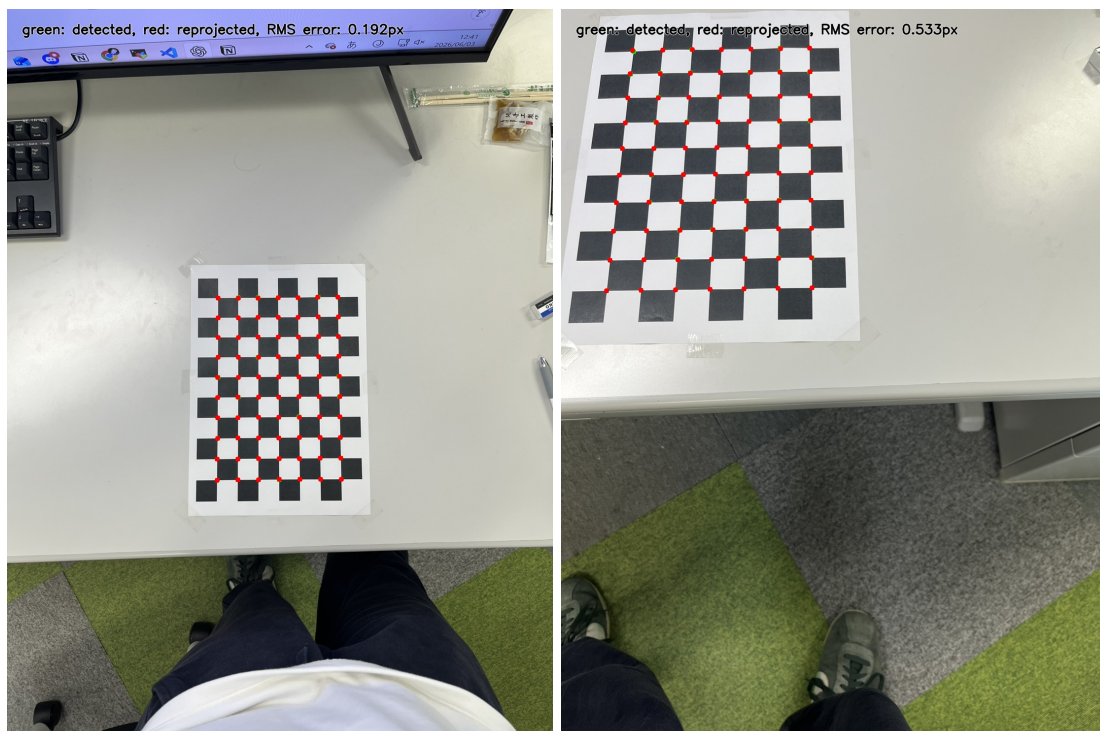


図5 Set_D_15 の最小誤差画像（左: 2.jpg, 0.192 px）と最大誤差画像（右: 12.jpg, 0.533 px）

5. 考察

まず、3枚のみを用いた Set_A_3 では RMS 誤差が最も小さくなった。しかし、この結果だけから Set_A_3 が最も良いキャリブレーションであるとは判断できない。RMS 誤差は、使用した画像に対して推定されたカメラパラメータが検出点と再投影点のずれをどの程度小さくできているかを表す値であり、別の撮影条件での画像に対しても正確であることを直接保証するものではない。また、チェッカーボードを遠くから撮影すると、画像上でチェッカーボードが小さく写るため、ピクセル単位の再投影誤差は小さく評価されやすい。そのため、遠距離、正面、画像中心付近の画像では RMS 誤差が小さく見えやすく、この点からも RMS 誤差だけで精度を判断することは適切ではない。

実際に、Set_A_3 では焦点距離が $f_x = 1235.763$ 、 $f_y = 1234.375$ と推定されており、7枚以上を用いた他のセットと比べて大きい値になっている。これは、Set_A_3 に含まれる画像が3枚と少ないことに加え、チェッカーボードを上から撮影し、対象物が画像中心付近に配置された画像に偏っていたためだと考えられる。カメラキャリブレーションでは、チェッカーボードをさまざまな角度や位置で撮影することで、焦点距離、主点、歪み係数を決めるための情報が増える。しかし、Set_A_3 のように、チェッカーボードの姿勢変化が小さく、対象物が画像中心付近に集中している場合、奥行き方向の変化や画像周辺部の歪みに関する情報が不足する。その結果、内部パラメータ

を決定するための制約が弱くなり、使用した 3 枚の画像に対しては小さい再投影誤差を与えるものの、焦点距離が他のセットと異なる値として推定された可能性がある。

一方、Set_B_7、Set_C_11、Set_D_15 では、焦点距離はおよそ 1140–1146 px の範囲に収まっており、Set_A_3 と比べて大きな変動が見られなかった。このことから、チェッカーボードの位置や角度の変化を含めることで、内部パラメータを決めるための条件が増えたと考えられる。

歪み係数についても、Set_A_3 では他のセットと比べて大きな値が推定された。特に半径方向歪み係数を見ると、Set_A_3 では $k_1 = 0.265$ 、 $k_2 = -1.831$ 、 $k_3 = 4.219$ であったのに対し、Set_D_15 では $k_1 = 0.098$ 、 $k_2 = -0.260$ 、 $k_3 = 0.274$ となった。この差は、Set_A_3 ではチェッカーボードが画像中心付近に集中しており、画像周辺部における歪みを十分に観測できなかったためと考えられる。歪み係数は主に画像中心から離れた点の変形によって制約されるため、中心付近の画像だけでは焦点距離と歪み係数の組み合わせに自由度が残りやすい。その結果、Set_A_3 では小さい RMS 誤差を保ちながらも、歪み係数が大きめに推定された可能性がある。一方、Set_B_7 以降では周辺部や傾いた姿勢の画像が含まれるため、歪みの影響を反映でき、Set_A_3 よりも極端でない係数になったと考えられる。また、接線方向歪みを表す p_1 、 p_2 はいずれのセットでも 10^{-4} 程度であり、半径方向歪みに比べて影響は小さいと考えられる。

ただし、画像枚数を増やせば RMS 誤差が必ず小さくなるわけではない。Set_D_15 では最も多い 15 枚の画像を用いたにもかかわらず、RMS 誤差は 0.425 px と 4 つのセットの中で最も大きかった。この理由として、画像数が増えたことで、チェッカーボードの傾きが大きい画像や、画像周辺部に配置された画像も含まれるようになり、検出点のわずかな誤差やレンズ歪みの影響が RMS 誤差に反映されやすくなったことが考えられる。つまり、Set_D_15 の RMS 誤差が大きいことは、必ずしもキャリブレーション精度が悪いことを意味しない。むしろ、多様な条件の画像を含めた結果、より広い撮影条件に対して妥当なパラメータを推定している可能性がある。

したがって、今回の結果からは、RMS 誤差の大小だけでキャリブレーションの良し悪しを判断するのではなく、推定された内部パラメータの安定性も併せて評価する必要があると分かる。Set_A_3 は RMS 誤差は小さいが、画像枚数が少なく、さらに上から撮影した画像や画像中心付近に対象物を配置した画像に偏っている。そのため、撮影条件の不足による過適合の可能性もある。一方、Set_B_7 以降では焦点距離が近い値に収束しているため、Set_A_3 よりも信頼性の高いキャリブレーション結果であると考えられる。

正確なキャリブレーションを行うためには、チェッカーボード全体が画像内に収まり、ぼけや反射が少ない画像を用いることが重要である。また、正面または上からの画像だけでなく、上下左右に傾けた画像や、画像中心だけでなく周辺部に配置した画像を含める必要がある。これにより、焦点距離や歪み係数を推定するための制約が増え、特定の撮影条件に偏りにくい内部パラメータを得ることができる。

今回の実験では、全てのセットで RMS 誤差は 1 px 未満であり、交点検出も成功している。そのため、全体としてはキャリブレーションに使用可能な画像品質であったと考えられる。ただし、最

も信頼できる結果を選ぶ際には、RMS 誤差が最小である Set_A_3 ではなく、画像枚数と撮影条件の多様性があり、かつ内部パラメータが安定している Set_B_7 以降の結果を重視するべきである。

6. まとめ

OpenCV を用いて、4 種類の画像セットに対してカメラキャリブレーションを行った。チェッカーボードの 1 マスは 24 mm、交点数は 7×10 として設定した。再投影誤差は 0.218 px から 0.425 px の範囲であり、すべて 1 px 未満であった。画像枚数が少ない場合は誤差が小さく見えることがあるが、推定値の安定性を考えると、複数の角度・位置を含む十分な枚数の画像を使うことが重要である。

付録: 使用したコード

本課題でカメラキャリブレーションに使用した Python コードを以下に示す。

```
1 from pathlib import Path
2 import csv
3 import json
4
5 import cv2
6 import numpy as np
7
8
9 PATTERN_SIZE = (7, 10)
10 SQUARE_SIZE_MM = 24.0
11 IMAGE_ROOT = Path("images")
12 OUTPUT_ROOT = Path("calibration_outputs")
13
14
15 def image_sort_key(path):
16     try:
17         return int(path.stem)
18     except ValueError:
19         return path.stem
20
21
22 def object_points():
23     cols, rows = PATTERN_SIZE
24     points = np.zeros((rows * cols, 3), np.float32)
25     grid = np.mgrid[0:cols, 0:rows].T.reshape(-1, 2)
26     points[:, :2] = grid * SQUARE_SIZE_MM
27     return points
28
29
30 def detect_corners(image_path):
31     gray = cv2.imread(str(image_path), cv2.IMREAD_GRAYSCALE)
32     if gray is None:
33         raise FileNotFoundError(image_path)
34
35     found, corners = cv2.findChessboardCornersSB(
36         gray,
37         PATTERN_SIZE,
38         cv2.CALIB_CB_NORMALIZE_IMAGE,
39     )
40     if not found:
41         return gray.shape[::-1], None
42
43     return gray.shape[::-1], corners.astype(np.float32)
44
45
46 def draw_reprojection(image_path, detected, projected, error_px, output_path):
47     image = cv2.imread(str(image_path))
48     projected = projected.reshape(-1, 2)
49     detected = detected.reshape(-1, 2)
50
51     for detected_point, projected_point in zip(detected, projected):
52         dx, dy = detected_point
53         px, py = projected_point
54         dxy = (int(round(dx)), int(round(dy)))
55         pxy = (int(round(px)), int(round(py)))
56         cv2.line(image, dxy, pxy, (0, 255, 255), 2)
57         cv2.circle(image, dxy, 5, (0, 255, 0), -1)
58         cv2.circle(image, pxy, 5, (0, 0, 255), -1)
59
60     cv2.putText(
61         image,
62         f"green: {detected}, red: {projected}, RMS error: {error_px:.3f}px",
```

```

63         (30, 50),
64         cv2.FONT_HERSHEY_SIMPLEX,
65         0.9,
66         (255, 255, 255),
67         4,
68         cv2.LINE_AA,
69     )
70     cv2.putText(
71         image,
72         f"green: {detected}, red: {reprojected}, RMS error: {error_px:.3f}px",
73         (30, 50),
74         cv2.FONT_HERSHEY_SIMPLEX,
75         0.9,
76         (0, 0, 0),
77         2,
78         cv2.LINE_AA,
79     )
80
81     output_path.parent.mkdir(parents=True, exist_ok=True)
82     cv2.imwrite(str(output_path), image)
83
84
85     def calibrate_set(set_dir):
86         objp = object_points()
87         objpoints = []
88         imgpoints = []
89         used_images = []
90         image_size = None
91
92         for image_path in sorted(set_dir.glob("*.jpg"), key=image_sort_key):
93             size, corners = detect_corners(image_path)
94             if corners is None:
95                 print(f"Warning: {corners} not found in {image_path}")
96                 continue
97             image_size = size
98             objpoints.append(objp)
99             imgpoints.append(corners)
100             used_images.append(image_path)
101
102         if len(imgpoints) < 3:
103             raise RuntimeError(f"{set_dir.name}: at least 3 valid images are required")
104
105         rms, camera_matrix, dist_coeffs, rvecs, tvecs = cv2.calibrateCamera(
106             objpoints,
107             imgpoints,
108             image_size,
109             None,
110             None,
111         )
112
113         per_image = []
114         plot_dir = OUTPUT_ROOT / set_dir.name / "reprojection_plots"
115         for image_path, obj, corners, rvec, tvec in zip(
116             used_images, objpoints, imgpoints, rvecs, tvecs
117         ):
118             projected, _ = cv2.projectPoints(obj, rvec, tvec, camera_matrix, dist_coeffs)
119             residuals = corners.reshape(-1, 2) - projected.reshape(-1, 2)
120             distances = np.linalg.norm(residuals, axis=1)
121             image_rms = float(np.sqrt(np.mean(distances**2)))
122             image_mean = float(np.mean(distances))
123             image_max = float(np.max(distances))
124
125             plot_path = plot_dir / f"{image_path.stem}_reprojection.jpg"
126             draw_reprojection(image_path, corners, projected, image_rms, plot_path)
127
128             per_image.append(
129                 {
130                     "image": image_path.name,
131                     "rms_px": image_rms,

```

```

132         "mean_px": image_mean,
133         "max_px": image_max,
134         "plot": str(plot_path),
135     }
136 )
137
138 return {
139     "set": set_dir.name,
140     "num_images": len(used_images),
141     "image_size": image_size,
142     "opencv_rms_px": float(rms),
143     "camera_matrix": camera_matrix,
144     "dist_coeffs": dist_coeffs.ravel(),
145     "per_image": per_image,
146 }
147
148
149 def save_results(results):
150     OUTPUT_ROOT.mkdir(parents=True, exist_ok=True)
151
152     summary_path = OUTPUT_ROOT / "calibration_summary.csv"
153     with summary_path.open("w", newline="", encoding="utf-8") as handle:
154         writer = csv.writer(handle)
155         writer.writerow(
156             [
157                 "set",
158                 "num_images",
159                 "opencv_rms_px",
160                 "mean_image_rms_px",
161                 "max_image_rms_px",
162                 "fx",
163                 "fy",
164                 "cx",
165                 "cy",
166                 "k1",
167                 "k2",
168                 "p1",
169                 "p2",
170                 "k3",
171             ]
172         )
173     for result in results:
174         matrix = result["camera_matrix"]
175         dist = result["dist_coeffs"]
176         image_rms_values = [item["rms_px"] for item in result["per_image"]]
177         writer.writerow(
178             [
179                 result["set"],
180                 result["num_images"],
181                 f"{result['opencv_rms_px']:.6f}",
182                 f"{np.mean(image_rms_values):.6f}",
183                 f"{np.max(image_rms_values):.6f}",
184                 f"{matrix[0,0]:.6f}",
185                 f"{matrix[1,0]:.6f}",
186                 f"{matrix[0,1]:.6f}",
187                 f"{matrix[1,1]:.6f}",
188                 f"{dist[0]:.8f}",
189                 f"{dist[1]:.8f}",
190                 f"{dist[2]:.8f}",
191                 f"{dist[3]:.8f}",
192                 f"{dist[4]:.8f}",
193             ]
194         )
195
196     detail_path = OUTPUT_ROOT / "calibration_details.json"
197     json_ready = []
198     for result in results:
199         json_ready.append(
200             {

```

```

201         "set": result["set"],
202         "num_images": result["num_images"],
203         "image_size": result["image_size"],
204         "opencv_rms_px": result["opencv_rms_px"],
205         "camera_matrix": result["camera_matrix"].tolist(),
206         "dist_coeffs": result["dist_coeffs"].tolist(),
207         "per_image": result["per_image"],
208     }
209 )
210 detail_path.write_text(
211     json.dumps(json_ready, indent=2, ensure_ascii=False),
212     encoding="utf-8",
213 )
214
215 return summary_path, detail_path
216
217
218 def main():
219     set_dirs = sorted(path for path in IMAGE_ROOT.glob("Set_*") if path.is_dir())
220     if not set_dirs:
221         raise RuntimeError(f"No Set_* folders found under {IMAGE_ROOT}")
222
223     results = [calibrate_set(set_dir) for set_dir in set_dirs]
224     summary_path, detail_path = save_results(results)
225
226     print(f"Pattern size: {PATTERN_SIZE[0]}x{PATTERN_SIZE[1]} inner corners")
227     print(f"Square size: {SQUARE_SIZE_MM:.1f}mm")
228     print(f"Summary: {summary_path}")
229     print(f"Details: {detail_path}")
230     print()
231
232     for result in results:
233         matrix = result["camera_matrix"]
234         dist = result["dist_coeffs"]
235         image_rms_values = [item["rms_px"] for item in result["per_image"]]
236         print(result["set"])
237         print(f"Images: {result['num_images']}")
238         print(f"OpenCV RMS: {result['opencv_rms_px']:.6f}px")
239         print(f"mean image RMS: {np.mean(image_rms_values):.6f}px")
240         print(f"max image RMS: {np.max(image_rms_values):.6f}px")
241         print(
242             f"K: {matrix[0,0]:.6f}, fy={matrix[1,1]:.6f}, "
243             f"cx={matrix[0,2]:.6f}, cy={matrix[1,2]:.6f}"
244         )
245         print(
246             f"distortion: "
247             f"k1={dist[0]:.8f}, k2={dist[1]:.8f}, "
248             f"p1={dist[2]:.8f}, p2={dist[3]:.8f}, k3={dist[4]:.8f}"
249         )
250         print()
251
252 if __name__ == "__main__":
253     main()
254
255

```

また、使用画像をデータセットごとに一覧表示するために用いたコードを以下に示す。

```

1 from pathlib import Path
2
3 import cv2
4 import numpy as np
5
6
7 IMAGE_ROOT = Path("images")
8 OUTPUT_ROOT = Path("calibration_outputs/dataset_overviews")
9 THUMB_W = 240
10 THUMB_H = 320
11 LABEL_H = 36

```

```

12 MARGIN = 16
13 COLS = 5
14
15
16 def image_sort_key(path):
17     try:
18         return int(path.stem)
19     except ValueError:
20         return path.stem
21
22
23 def make_thumbnail(path):
24     image = cv2.imread(str(path))
25     if image is None:
26         raise FileNotFoundError(path)
27
28     h, w = image.shape[:2]
29     scale = min(THUMB_W / w, THUMB_H / h)
30     resized = cv2.resize(image, (int(w * scale), int(h * scale)))
31
32     tile = np.full((THUMB_H + LABEL_H, THUMB_W, 3), 245, dtype=np.uint8)
33     x = (THUMB_W - resized.shape[1]) // 2
34     y = (THUMB_H - resized.shape[0]) // 2
35     tile[y : y + resized.shape[0], x : x + resized.shape[1]] = resized
36
37     label = path.name
38     cv2.putText(
39         tile,
40         label,
41         (10, THUMB_H + 25),
42         cv2.FONT_HERSHEY_SIMPLEX,
43         0.7,
44         (0, 0, 0),
45         2,
46         cv2.LINE_AA,
47     )
48     return tile
49
50
51 def make_overview(set_dir):
52     files = sorted(set_dir.glob("*.jpg"), key=image_sort_key)
53     rows = (len(files) + COLS - 1) // COLS
54     canvas_h = MARGIN + rows * (THUMB_H + LABEL_H + MARGIN)
55     canvas_w = MARGIN + COLS * (THUMB_W + MARGIN)
56     canvas = np.full((canvas_h, canvas_w, 3), 255, dtype=np.uint8)
57
58     for index, path in enumerate(files):
59         row = index // COLS
60         col = index % COLS
61         x = MARGIN + col * (THUMB_W + MARGIN)
62         y = MARGIN + row * (THUMB_H + LABEL_H + MARGIN)
63         tile = make_thumbnail(path)
64         canvas[y : y + tile.shape[0], x : x + tile.shape[1]] = tile
65
66     OUTPUT_ROOT.mkdir(parents=True, exist_ok=True)
67     output_path = OUTPUT_ROOT / f"{set_dir.name}_overview.jpg"
68     cv2.imwrite(str(output_path), canvas)
69     print(output_path)
70
71
72 def main():
73     for set_dir in sorted(path for path in IMAGE_ROOT.glob("Set_*") if path.is_dir()):
74         make_overview(set_dir)
75
76
77 if __name__ == "__main__":
78     main()

```